

Trabajo seminario ES_206. Rust en sistemas críticos

Eloy Rubio

Contexto y presentación

Giovanni Asproni entrevista a Florian Gilcher, co-fundador y director general de Ferrous Systems, empresa creadora de Ferrocene, un compilador Rust cualificado para sistemas críticos. Florian lleva trabajando con Rust desde 2013, es co-fundador de la Rust Foundation y formador del lenguaje desde 2015. Ya había aparecido en el episodio 279 del podcast, cuando representaba al equipo de comunidad del proyecto Rust.

Qué son los sistemas mission y safety critical

Florian define los sistemas mission critical como aquellos cuyo fallo tiene consecuencias graves, principalmente financieras: un datacenter que cae, un sistema hospitalario que deja de operar. Los safety critical son aquellos donde un fallo puede costar vidas: software de frenos, aviónica, dispositivos médicos.

Sin embargo, matiza que esta distinción es cada vez más difícil de mantener. Un datacenter de AWS que cae puede desencadenar fallos en cascada en sistemas de seguridad que no habían anticipado esa dependencia. Y pone un ejemplo concreto: un fallo de memoria en la infraestructura de Google fue la causa de una interrupción de servicio, algo que, en su opinión, no habría ocurrido con Rust. La hiperconectividad hace que sistemas que antes se consideraban puramente financieros puedan tener implicaciones para vidas humanas.

Los estándares de la industria

Una de las diferencias fundamentales entre el mundo safety critical y el resto de la ingeniería del software es la existencia de estándares formales. Florian menciona los principales: IEC 61508 para industria general, ISO 26262 para automoción, DO-178 para aviónica y MISRA como guía de codificación para C y C++. Estos estándares son el producto de consorcios industriales que llevan décadas debatiendo qué constituye buena práctica en software crítico.

Los estándares no son prescriptivos al cien por cien. Definen recomendaciones y niveles de rigor, en automoción los niveles se llaman ASIL A, B, C y D, de menor a mayor criticidad, y para cada nivel establecen qué prácticas se esperan. Por ejemplo, ASIL B exige cobertura de sentencias en los tests, mientras que ASIL D exige MCDC, cobertura de condiciones y decisiones múltiples, considerablemente más exigente. Lo que hace especial a estos estándares, según Florian, es que, si un equipo decide apartarse de una recomendación, puede hacerlo, pero tiene que argumentarlo ante un evaluador independiente. Esa separación entre quien implementa y quien valida es estructural y obligatoria. En el mundo mission critical esa práctica existe de forma informal, pero no con el mismo nivel de formalización.

Un detalle relevante que menciona: el ISO 26262 recomienda explícitamente usar un lenguaje de tipado estático si está disponible, lo que abre formalmente la puerta a Rust en Automoción.

Alejandro González

¿Por qué usar Rust?

Rust nació en Mozilla, una empresa de navegadores web. No en aviónica ni en automoción. Y sin embargo resulta ser una elección muy natural para sistemas críticos. ¿Por qué?

- Primero, la promesa central: memory safety en tiempo de compilación. Sin garbage collector, sin runtime. El compilador garantiza que no hay dangling pointers, ni buffer overflows, ni data races en concurrencia. Eso elimina categorías enteras de bugs que en C hay que cazar con herramientas externas caras.
- Segundo, no tiene runtime. Un programa Rust simple es equivalente en comportamiento a un programa C: ejecuta en el procesador sin capas intermedias. Eso es crítico en sistemas embebidos donde no tienes un sistema operativo debajo.
- Tercero el sistema de ownership y borrowing. Ownership, Cada recurso (normalmente memoria) tiene un propietario único. Borrowing Rust permite acceder a datos mediante referencias seguras. Si compila, hay una garantía muy sólida de que el programa es correcto en cuanto a gestión de memoria.
- Y cuarto: interopera bien con C. Puedes mezclar código Rust y C, llamar a librerías existentes, y hacer una migración incremental función a función. No tienes que reescribir todo de golpe.

El compilador de Ferrocene

Ahora bien, para usar Rust en un sistema certificado no basta con que el lenguaje sea bueno. El compilador también tiene que estar cualificado. Y aquí entra Ferrocene, el proyecto de Ferrous Systems.

Ahora bien, para usar Rust en un sistema certificado no basta con que el lenguaje sea bueno. El compilador también tiene que estar cualificado. Y aquí entra Ferrocene, el proyecto de Ferrous Systems.

Ferrocene es un downstream del compilador Rust oficial. No es un fork; toman los cambios del upstream cada noche y sobre eso añaden capas de validación: tests trazados a la especificación del lenguaje, ejecución en microcontroladores reales como Cortex-M o RISC-V, y soporte a largo plazo por versión —porque en safety critical no puedes decirle a un cliente "actualiza cada seis semanas o no te damos soporte".

Una contribución importante es que Ferrocene desarrolló la primera especificación formal del lenguaje Rust, que el propio proyecto Rust ha aceptado como oficial. Eso elimina un argumento que durante una década bloqueaba a Rust en entornos regulados.

La parte comercial es interesante: el código fuente es open-source, el proceso es público y visible en GitHub. Pero la versión cualificada requiere una empresa detrás que pueda asumir responsabilidad contractual y responder a incidencias en plazo. Eso no puede hacerlo un proyecto de voluntarios, por muy buena calidad que tenga el código.

Andrea Ivanov

¿Criterios para pasar de un lenguaje a Rust?

Hay dos criterios principales que Florian pone sobre la mesa a la hora de reescribir un proyecto en Rust.

1 - ¿Tienes algún problema por el que quieras cambiar?

- Proyectos fracasan: el motor es estar a la moda.
- Si algo funciona, no lo cambies.

Se puntualiza que muchos proyectos fracasan porque el motor de cambiar de lenguaje fue estar a la moda. Si algo ya funcionaba y no supone una necesidad el implementar las características de Rust, no te comas la cabeza.

2 - ¿Tengo los recursos?

- ¿Tengo a los ingenieros, tiempo, etc. suficiente para involucrarme?
- Empieza por algo pequeño.

Tengo los ingenieros, el proyecto, el tiempo... para involucrarme en un proyecto tan grande como reestructurar en otro lenguaje. Aconseja empezar por algo pequeño y si tiene éxito, intentas algo más grande.

Se puntualiza tener disponibilidad de tiempo, hay una relación directa con el tiempo invertido y el dinero perdido. A no ser que no tengas alternativa.

Ventajas

- Si no llegas a tiempo, ya tienes algo hecho.

Suele haber una advertencia con el reescribir el código de una aplicación que ya funciona por la cantidad de recursos y tiempo que conlleva, pero Florian aquí nos cuenta que la ventaja es que, si no llegas a tiempo o no funciona, tendrás la versión antigua que sí lo hace.

- Interoperabilidad entre C y Rust.

En su momento, en el proyecto de firefox, para que Rust fuera útil, necesitaba poder comunicarse con C. Esto facilita mucho el trabajo ya que la interoperabilidad es crucial a la hora de valorar la traducción. Por ejemplo, librsvg, una biblioteca para renderizar imágenes SVG, no borraron todo el código, fueron función por función.

- No supone un cambio generacional (para viejos programadores)

Al tener esta relación con C, se pueden aprovechar mucho más los conocimientos técnicos.

Inconvenientes

- Pérdida de sabiduría técnica (para nuevos programadores)

No es lo mismo tener que desarrollar código que reescribirlo, es un ejercicio mayor que te proporciona mucho más aprendizaje.

- Curva de aprendizaje muy empinada por dos términos ownership y borrowing.

Se ha aplanado por la mejora de las descripciones del compilador.

Conclusión

- Próximo lenguaje estándar para seguridad de memoria.

Florian se imagina Rust como el próximo lenguaje estándar para estas áreas de seguridad de memoria. Aun así, como todo, los extremos no son buenos. Es importante que siga habiendo una competencia sana de lenguajes y sigan creciendo otros para que Rust no se vuelva perezoso.